

SG-ECREDAC — Sistema de Gestão E-CREDAC

Sistema web interno da **RNV Consultoria** para validar dados fiscais brasileiros, apurar rateio de custos e gerar o arquivo **E-CREDAC** (Sistema Eletrônico de Gerenciamento do Crédito Acumulado de ICMS), além das fichas de acompanhamento exigidas no processo.

Este README descreve o que o sistema faz, como ele está organizado no código e como subir o ambiente localmente.

Índice

- [O que o sistema resolve](#)
 - [Fluxo de trabalho](#)
 - [Tecnologias](#)
 - [Requisitos](#)
 - [Instalação](#)
 - [Configuração](#)
 - [Como executar](#)
 - [Estrutura do projeto](#)
 - [Aplicações Django](#)
 - [Funcionalidades](#)
 - [Permissões](#)
 - [Banco de dados](#)
 - [Rotas principais](#)
 - [Arquitetura da validação](#)
 - [Uploads e limites](#)
 - [Documentação adicional](#)
 - [Troubleshooting](#)
 - [Notas para desenvolvedores](#)
 - [Licença](#)
-

O que o sistema resolve

O **E-CREDAC** é o arquivo eletrônico usado para demonstrar o crédito acumulado de ICMS. Antes de gerá-lo, a empresa precisa:

1. Ter cadastro fiscal consistente (empresa, produtos, participantes).
2. Conferir o **SPED Fiscal** com as **NF-e (XML)** e, quando houver, **DUE** e outros modelos (CT-e, energia, comunicação).
3. Processar o **Bloco K** (produção e insumos) e/ou a **planilha de custo**.
4. Preencher as **fichas de controle de custos**.
5. Exportar o arquivo E-CREDAC com os registros do Bloco 5 (por exemplo 5010, 5015 e 5020).

O SG-ECREDAC concentra esse fluxo em uma aplicação Django, com tela de login, dashboard, histórico de alterações e painel administrativo.

Fluxo de trabalho

O uso típico segue esta ordem:

Login

- Cadastro da empresa (manual ou via SPED)
- Cadastro / conferência de produtos (bloco 0200)
- Upload de movimentações (SPED + XML, DUE, outros modelos)
- Painel de NF-e / outros modelos em andamento
- Edição e validação dos dados extraídos
- Rateio (Bloco K e/ou planilha de custo)
- Fichas de controle
- Gerar arquivo E-CREDAC

Cada etapa depende da anterior. Por exemplo, a geração do arquivo só considera empresas com **movimentação concluída** para o mês informado (`ValidacaoDataConcluida`).

Tecnologias

Camada	Tecnologia
Backend	Django 5.2, Python 3.10+
Banco	PostgreSQL 12+
Dados	pandas, openpyxl
XML	<code>xml.etree.ElementTree</code>
Admin	django-jazzmin
Frontend	HTML, CSS, JavaScript, Bootstrap 5, Font Awesome
Ambiente	<code>python-dotenv</code> (arquivo <code>.env</code>)

Dependências oficiais: `requirements.txt`.

Requisitos

- Python 3.10 ou superior
 - PostgreSQL 12 ou superior
 - pip e Git
 - Ambiente virtual (`venv`)
-

Instalação

1. Clone e ambiente virtual

```
git clone <url-do-repositorio>
cd RNV_ECREDAC

# Windows
python -m venv venv
venv\Scripts\activate

# Linux / macOS
python3 -m venv venv
source venv/bin/activate
```

2. Dependências

```
pip install -r requirements.txt
```

O `settings.py` usa `python-dotenv`. Se o pip não instalar esse pacote pelo `requirements.txt`, instale à parte:

```
pip install python-dotenv
```

Configuração

Banco PostgreSQL

No PostgreSQL, crie o banco (o nome pode ser outro, desde que bata com o `.env`):

```
CREATE DATABASE ecredac;
```

O usuário precisa ter permissão nesse banco. Em ambientes locais costuma-se usar o usuário `postgres`.

Arquivo `.env`

O projeto **não** lê usuário e senha direto no `settings.py`. Crie um `.env` na raiz (esse arquivo já está no `.gitignore`):

```
DB_NAME=ecredac
DB_USER=postgres
DB_PASSWORD=sua_senha
DB_HOST=localhost
DB_PORT=5432
```

O Django carrega essas variáveis em `project/settings.py` via `load_dotenv()`.

Migrações e superusuário

```
python manage.py migrate
python manage.py createsuperuser
```

O superusuário acessa o Django Admin (`/admin/`). As telas do sistema (home, cadastro, validação etc.) usam o login em `/login/`.

Crie também a pasta de logs, se ainda não existir:

```
mkdir logs
```

Erros do Django são gravados em `logs/django.log`.

Como executar

Desenvolvimento

```
python manage.py runserver
```

- Sistema: <http://127.0.0.1:8000/> (redireciona para o login)
- Admin: <http://127.0.0.1:8000/admin/>

Na tela de login é possível escolher o modo de acesso (sistema ou admin).

Comandos úteis

```
python manage.py migrate
python manage.py createsuperuser
python manage.py collectstatic
python manage.py shell
```

Estrutura do projeto

```
RNV_ECREDAC/
├── accounts/           # Login e redirecionamento
├── cadastro/          # Empresas, produtos e regras de código
├── validacao/         # Upload, parse e validação fiscal
└── metodo_rateio/     # Bloco K (SPED) e planilha de custo
```

```

|— gerar_arquivo/      # Montagem do arquivo E-CREDAC
|— gerar_fichas/       # Fichas 1 a 6 de controle de custos
|— historico/          # Auditoria de alterações nas telas
|— home/               # Dashboard e permissões por usuário
|— help/               # Página de ajuda
|— regras_jobs/        # Lógica de validação de regras de código (não é app
Django)
|— project/            # settings, urls, wsgi, asgi
|— templates/          # Templates globais (base.html fica em home/)
|— static/             # CSS, JS e imagens globais
|— docs/               # Manuais e relatórios técnicos
|— logs/               # django.log
|— manage.py
|— requirements.txt
|— .env                # credenciais locais (não versionar)

```

Cada app Django, em geral, separa **urls**, **views**, **models**, **services** e **templates**. A validação ainda tem a pasta **parser/** para extração de SPED, XML e planilha DUE.

Aplicações Django

accounts

Login com **CustomLoginView**. Depois do login, o usuário vai para **/home/** ou para **/admin/**, conforme o modo escolhido.

home

Dashboard inicial e modelo **Permissions**, que liga cada usuário às telas que ele pode ver (cadastro, movimentação, rateio, fichas, gerar arquivo). O context processor **home.context_processors.permissoes** injeta essas flags em todos os templates.

cadastro

- **Empresa:** cadastro manual ou extração do SPED (registro 0000 / dados do contribuinte). Campos fiscais usados depois no arquivo E-CREDAC (**ladca**, **cod_ver**, **cod_fin**, IE, município, opção de crédito outorgado etc.).
- **Produtos:** itens do bloco **0200**, modelo **Cadastro_itens_sped**.
- **Regras de lançamento:** prefixo, sufixo, tamanho, formato e caractere. As regras ficam no banco (**Regra / EmpresaRegra**) e são aplicadas na extração do SPED via **regras_jobs.validacao**.

validacao

Núcleo do sistema. Processa arquivos e guarda o resultado para conferência.

Tipo	O que entra	O que sai
NF-e	SPED .txt + XML da NF-e	Participantes (0150), notas, itens, catálogo 0200

Tipo	O que entra	O que sai
DUE	Planilha Excel de Declaração Única de Exportação	Complemento das notas de exportação
Outros modelos	SPED (C500/C590, D100/D190, D500/D590)	Energia, transporte (CT-e) e comunicação

O painel **NF-es em andamento** e **Outros modelos em andamento** mostra o progresso (**ValidacaoStatus**). Quando a conferência termina, a data/mês entra em **ValidacaoDataConcluida**.

metodo_rateio

- **Bloco K via SPED:** produção própria **K230/K235** e industrialização **K250/K255**.
- **Planilha de custo:** Excel com centros de custo, valores e ICMS passível de crédito.

Há telas de upload, análises em andamento e visualização em tabela (DataFrame) com edição.

gerar_arquivo

Monta o arquivo texto E-CREDAC a partir dos dados já validados da empresa e do mês. Só gera se existir movimentação **concluída** para aquele período. Detalhe dos registros 5010/5015/5020:

[docs/relatorio_registros_5010_5015_5020.md](#).

gerar_fichas

Menu de fichas de controle de custos, agrupadas assim:

Grupo	Tema	Exemplos
1	Insumos	1A materiais, 1B terceirização, 1C energia, 1D telecom, 1E transportes
2	Processo produtivo	2A–2C elaboração, 2D transporte, 2E GGF, 2F–2G produção conjunta
3	Produtos acabados e revenda	3A, 3B, 3C
4	Rateios	4A energia, 4B índice de alocação, 4C GGF
5	Demonstrativos auxiliares	ficha técnica, participantes, enquadramento legal etc.
6	Geração de crédito acumulado	alíquotas, ZFM, exportação, transporte

historico

Registra alteração campo a campo (valor antigo/novo, usuário, tela, mês/ano do SPED). Telas rastreadas: movimentação, método de rateio, cadastro e gerar arquivo.

pendencias e help

Telas de apoio: pendências do processo e ajuda ao usuário.

regras_jobs (módulo, não é app)

Usado pelos parsers para aplicar as regras de código de produto cadastradas em `cadastro`. Não entra em `INSTALLED_APPS`.

Funcionalidades

Cadastro

- Empresa manual ou via arquivo SPED
- Produtos extraídos do bloco 0200, com edição e exclusão
- Regras de código de lançamento por empresa
- Empresa pode ser desativada (`status`) sem sumir do histórico

Processamento de arquivos

- SPED Fiscal (`.txt`): participantes, notas, produtos, impostos, Bloco K
- XML de NF-e (arquivos avulsos ou pasta)
- Planilha DUE
- Outros modelos do SPED (energia C500/C590, transporte D100/D190, comunicação D500/D590)
- Planilha Excel de custo para rateio
- Upload de muitos arquivos no mesmo POST (ver [Uploads e limites](#))

Validação e análise

- Validação por empresa, mês e tipo de documento
- Status da nota (ex.: com produto / sem produto)
- Classificação de item (com nota, sem nota, sem cadastro)
- Cruzamento SPED × XML
- Edição inline nas tabelas, filtros e exportação Excel

Saídas

- Arquivo E-CREDAC
 - Fichas 1 a 6
 - Relatórios Excel das telas de DataFrame
 - Histórico de auditoria
-

Permissões

O modelo `home.Permissions` controla o que aparece no menu para cada usuário:

- Cadastro
- Movimentações (validação)
- Método de rateio
- Gerar fichas

- Gerar arquivo

A configuração é feita no Django Admin. Sem permissão, o item some da barra superior (`home/templates/base.html`).

Banco de dados

O banco é **PostgreSQL**. Os modelos principais:

App	Modelos	Função
cadastro	Empresa, Regra, EmpresaRegra, Cadastro_itens_sped	Cadastro mestre
validacao	Participantes, Notas_participantes, Produtos_notas	NF-e / mercadorias
validacao	ValidacaoStatus, ValidacaoDataConcluida	Painel e conclusão por período
validacao	RegistroEnergiaC500/C590, RegistroTransporteD100/D190, RegistroComunicacaoD500/D590	Outros modelos
metodo_rateio	ItensProduzidos230, InsumosUsados235, ItensProduzidos250, InsumosUsados255	Bloco K
metodo_rateio	analise_k23x, analise_k25x, PlanilhaCusto	Análises e planilha
historico	Historico	Auditoria
home	Permissions	ACL por tela
gerar_fichas	Codigos_lancamentos e modelos das fichas	Fichas e códigos

Relacionamento típico da NF-e:

```

Empresa
├── Participantes (bloco 0150)
│   ├── Notas_participantes
│   │   └── Produtos_notas

```

Rotas principais

Prefixos definidos em `project/urls.py`:

URL	Função
/	Redireciona para login
/login/	Login
/logout/	Logout

URL	Função
/admin/	Django Admin (Jazzmin)
/home/	Dashboard
/cadastro_empresa/	Nova empresa, edição, exclusão, cadastro via SPED
/cadastro_produto/	Cadastro e edição de produtos
/cadastro_listagem/	Listas de empresas e produtos
/regras_cod_lan/	Regras de código de lançamento
/upload/sped_xml/	Upload SPED + XML
/upload/due/	Upload DUE
/validacao_dados/painel_de_controle/	NF-es em andamento
/validacao_dados/outros_modelos_em_andamento/	Outros modelos em andamento
/validacao_dados/movimentacoes/	Tabela de dados NF-e
/validacao_dados/view-dados-servicos/	Tabela de outros modelos
/sped/	Upload e análise do Bloco K
/planilha/	Upload e análise da planilha de custo
/menu_fichas/	Menu das fichas
/fichas1/ ... /fichas6/	Telas de cada ficha
/gerarArquivo/gerar_arquivo/	Geração do arquivo E-CREDAC
/historico/	Histórico de alterações
/ajuda/	Ajuda

Arquitetura da validação

A app **validacao** está dividida em camadas. Vale seguir essa separação em código novo:

```
Upload (views/uploads)
  → Processamento (services/nfe ou services/outros_modelos)
    → Parser (parser/nfe ou parser/outros_modelos)
      → Models
      → DataFrame (salvar / exportar)
    → Tela de conferência (views + templates + JS)
```

Arquivos de extração atuais:

Arquivo	Responsabilidade
<code>validacao/parser/nfe/extract_sped.py</code>	Classe <code>SPEDProcesses</code> — SPED de mercadorias
<code>validacao/parser/nfe/extract_xml.py</code>	XML da NF-e
<code>validacao/parser/nfe/extract_planilhaDue.py</code>	Planilha DUE
<code>validacao/parser/outros_modelos/extract_campoServicos.py</code>	C500, D100, D500 e registros filhos
<code>validacao/utis/normalizadores.py</code>	CNPJ, datas, decimais, texto
<code>metodo_rateio/utis/extract_blocok.py</code>	Bloco K (K230/K235, K250/K255)
<code>cadastro/utis/extract_sped.py</code>	Extração para cadastro de empresa

Serviços de persistência e exportação ficam em `validacao/services/` (NF-e e outros modelos) e `metodo_rateio/services/`.

Uploads e limites

O sistema aceita muitos XMLs no mesmo envio. Em `project/settings.py`:

Setting	Valor	Significado
<code>DATA_UPLOAD_MAX_NUMBER_FILES</code>	2000	Arquivos por POST
<code>DATA_UPLOAD_MAX_MEMORY_SIZE</code>	100 MB	Tamanho total em memória
<code>FILE_UPLOAD_MAX_MEMORY_SIZE</code>	100 MB	Tamanho por arquivo em memória
<code>DATA_UPLOAD_MAX_NUMBER_FIELDS</code>	10000	Campos no formulário

Arquivos maiores que o limite em memória vão para disco temporário; o parse continua funcionando, só fica mais lento.

Documentação adicional

Na pasta `docs/`:

Arquivo	Conteúdo
<code>sg-ecredac_documentacao.md</code>	Documentação técnica expandida (arquitetura, fluxos, classes)
<code>MANUAL_DO_USUARIO.md</code>	Passo a passo operacional para o usuário fiscal
<code>relatorio_registros_5010_5015_5020.md</code>	Layout dos registros do Bloco 5 no arquivo gerado

Este README é o ponto de partida. Os arquivos em [docs/](#) aprofundam cálculos, layout de registro e uso da interface.

Troubleshooting

Não conecta no banco

Confira se o PostgreSQL está no ar e se o `.env` tem `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_HOST` e `DB_PORT`. Sem `.env`, essas variáveis ficam vazias.

Erro de migração

Com o venv ativado: `python manage.py migrate`. Se uma app nova não aparecer, confira `INSTALLED_APPS` em `project/settings.py`.

Pacote não encontrado

Ative o venv e rode `pip install -r requirements.txt`. O carregamento do `.env` depende de `python-dotenv`.

CSS/JS não carregam

Em desenvolvimento o Django serve `STATICFILES_DIRS`. Em produção: `python manage.py collectstatic`.

Upload estoura limite

Reduza a quantidade de XMLs por lote ou aumente os settings de upload. Logs em `logs/django.log`.

Gerar arquivo retorna 400

A empresa precisa ter validação **concluída** para o mês escolhido (`ValidacaoDataConcluida`), com participantes e itens SPED na mesma data.

Menu some para o usuário

Cadastre `Permissions` no Admin ligando o usuário às telas desejadas.

Notas para desenvolvedores

- Idioma da interface e do `LANGUAGE_CODE`: `pt-br`. Fuso: `America/Sao_Paulo`.
- Views novas devem seguir o padrão de classes (`django.views.View`) já usado nas apps.
- Não coloque regra de negócio pesada na view: use `services/` e `parser/`.
- Alterações visíveis ao usuário em movimentação, cadastro, rateio ou gerar arquivo devem ir para `historico`.
- Teste com SPED e XML reais da empresa antes de considerar a feature pronta: o layout varia (blocos, encoding, regras de código).
- `SECRET_KEY` e `DEBUG = True` em `settings.py` são adequados só para desenvolvimento. Em produção use variáveis de ambiente e `DEBUG = False`.
- Não commite `.env`, `venv/` nem `logs/*.log`.

Checklist rápido antes de alterar o núcleo fiscal:

1. Ler este README e, se for Bloco 5, o relatório em [docs/](#).
2. Localizar o parser/serviço correspondente (tabela da seção [Arquitetura da validação](#)).

3. Rodar o fluxo completo: upload → painel em andamento → edição → concluir → gerar arquivo.

Licença

Uso interno exclusivo da **RNV Consultoria**. Redistribuição somente com autorização expressa.

Última atualização: agosto de 2026